CS482 Software Project:
A Social Seven-Card Stud Gaming Platform

Shane Ford, Parmveer Basara, Ben Ackerman, and Denis Moroz

2026-11-09

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Hoang Bui
- Client title: Head of the mechanic department
- Client email address: hdbui@loyola.edu
- Client employer: Stack Overflow
- How you know the client: Our team was connected with the client for our CS482 Software Engineering project to develop a software solution based on the company's needs.

2 Project Description

2.1 Overview

Our project is to develop a web-based Seven-Card Stud poker application for a poker-chip company called Stack Overflow. The application will allow users to play Seven-Card Stud online using virtual poker chips rather than real money. The main goal is to create a functional and entertaining poker platform that promotes the Stack Overflow brand while encouraging users to return and continue playing.

The client wants the application to provide both guest and registered-user experiences. Guests will be able to play without creating an account, while registered users will be able to save their virtual chips, track statistics, create a profile, add friends, and use social features. The application will also provide a lobby where players can find active tables, create public or private tables, and play with friends. It will be for anyone, with no age limit, playing for fun.

Another purpose of the application is to help promote the client's poker-chip company. In the future, the platform may generate revenue through advertisements, including advertisements related to casinos and other relevant organizations. Ads can earn players more chips, since there is no real money. For the first version, however, the main priority is implementing a reliable and functional Seven-Card Stud game before focusing on advanced animations and other visual enhancements. For these enhancements, we were asked to use playwsop.com as a reference for its look and feel.

For deployment, it will run on localhost for now and then move on to the client's servers later.

2.2 Key Features

- Seven-Card Stud poker gameplay with virtual chips and no real-money gambling.
- Guest access for users who want to play without creating an account. They wont keep chips or stats.

- Registered user accounts with Google authentication.
- User profiles with usernames and profile pictures or avatars.
- Saved virtual-chip balances and player statistics.
- A casino-style lobby displaying available and active poker tables.
- Ability to create and join public or private poker tables.
- Configurable table settings, including betting limits and antes.
- Real-time multiplayer gameplay.
- Friend requests, friend lists, and the ability to see and join friends at tables.
- In-application messaging and table chat. Taunts will be allowed
- Player statistics such as wins, losses, win percentage, hands played, chips, and betting/folding behavior.
- Leaderboards based on statistics such as win rate and chip balance.
- Beginner assistance that can help users understand the strength of their poker hand.
- A tutorial explaining how Seven-Card Stud works.
- Practice games against computer-controlled bots with different difficulty levels.
- Administrator tools for monitoring games and chats, reviewing reports, and managing inappropriate users.
- Player reporting and moderation features.
- Turn timers of approximately 10–15 seconds to keep games moving.
- Removal of inactive players after repeated failures to participate.
- System announcements and notifications for events.
- Placeholder advertisements with the ability to expand advertising features later.
- Purple branding based around the client's company color.
- A Stack Overflow poker-chip company logo and consistent company branding throughout the application.
- Players can upload their own picture or choose a premade avatar

2.3 Why this Project is Interesting

This project is interesting because it combines several areas of computer science to create an interactive multiplayer gaming experience. Developing a Seven-Card Stud poker application presents unique challenges, such as implementing game rules, managing real-time player interactions, tracking statistics, and creating computer-controlled opponents. It is also an opportunity to gain hands-on experience with web development, databases, networking, and software engineering while working on a real-world project.

What makes this project enticing is the opportunity to build something beyond a traditional website. The application requires both technical functionality and an engaging user experience, allowing us to be creative while solving complex programming problems. This project is important because it demonstrates how software can bring people together through online gaming while also supporting a business by increasing brand awareness and creating opportunities for future growth. Additionally, the finished application could provide users with an accessible way to learn and enjoy poker without risking real money.

2.4 Areas of CS required

- Software Engineering: Requirements analysis, user stories, system design, version control, testing, and iterative development.
- Web Development: Creating the browser-based front-end and backend of the application.
- Database Systems: Storing user profiles, virtual-chip balances, friends, statistics, game history, reports, and other application data.
- Software Testing: Testing poker rules, game logic, user interactions, database operations, and multiplayer functionality.
- Authentication and Security: Supporting Google authentication, user permissions, administrator access, and account security.
- Algorithms and Data Structures: Managing the deck, dealing cards, evaluating poker hands, determining winners, and controlling game logic.
- Networking: Keeping the game state in sync for all players in real time.

2.5 Potential Concerns and Questions

One of our main concerns is the overall scope of the project. The client's complete vision includes multiplayer poker, accounts, social features, messaging, bots, statistics, leaderboards, moderation, advertisements, and other features. Completing all of these features at a high level of quality within the available development time may be difficult. We may need to prioritize the core gameplay and account functionality before implementing secondary features.

Real-time multiplayer gameplay may also be one of the more technically challenging areas. The game must keep each player's cards, bets, chip balances, turns, timers, and overall game state synchronized. We will need to determine the best architecture and technologies for handling these real-time interactions. Another concern is the development of poker bots. Creating basic bots may be achievable, but developing multiple realistic difficulty levels could significantly increase the complexity of the project. We would like instructor feedback on how advanced the bot functionality should be for the scope of a CS482 project.

We would also like feedback on whether features such as leaderboards, advertising, advanced animations, and some of the larger social features should be considered requirements for the initial version or treated as future enhancements. Our current plan is to prioritize correct Seven-Card Stud gameplay, virtual chips, user and guest access, authentication, the lobby and tables, multiplayer gameplay, basic profiles and statistics, and reliable real-time interaction before moving on to these additional features.

3 Requirements

3.1 Non-Functional Requirements

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
PE-01	Crashes	error handling	The site will not crash if the user does something unexpected.
PE-02	loading	performance	when making a action on the site it will respond quickly.
VD-01	UI	usability	the layout of the web app should easy to understand and help users get into game quickly
BT-01	Tutorial	usability	have a clean demo game for new users to understand the game
VD-02	colors	usability	adding color to differentiate different actions
VD-03	chat box	design	chat box in a corner to keep away from the main table
AM-01	report	usability	finding the report button or getting in contact to a admins should be easy

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
LT-01	View Lobby	3	As a player, I want to view available and active poker tables in the lobby, so that I can find a game to join.
LT-02	Join Table	1	As a player, I want to join an available poker table, so that I can participate in a game.
SF-01	Send Friend Request	2	As a registered user, I want to send friend requests to other players, so that I can connect with them.
SF-02	View Friends List	2	As a registered user, I want to view my friends list, so that I can find and connect with my friends.
SF-03	Receive Friend Requests	1	As a registered user, I want to accept or decline friend requests, so that I can manage my connections.
ST-01	Player Statistics	1	As a registered user, I want to view my wins, losses, and win percentage, so that I can track my performance over time.
PG-01	deal cards	2	As a player, I want to be dealt the right amount of cards faced down and up when it is the right time so that I can play the game.
PG-02	Gambling	1	As a player, I want to be able to gamble after 3rd card until the last card and to be able to fold, raise, check, or complete so that I can gamble
SF-04	remove friend	1	As a user, I want to be able to remove friend, so that I can leave someone I'm not friends with anymore.
LT-03	create table	3	As a user, I want to be able to create a table and invite friends or let randoms join, so that I can play whenever and with friends.
SF-05	chat with friend	3	As a user, I want to be able to talk to just my friends so that our conversation is not public.
AM-01	reporting user	3	As a user, I want to be able to report a user and be able to talk to admins about issues, so that the game is a pleasant experience.
ST-02	leaderboard	2	As a user, I want to be able to see other's score, so that I can compare mine to the top players
SF-06	Chat room	3	As a player, I want to be able to type to other players, so that I can make friends
BT-01	bots	5	As a player, I want bots to play when I am alone in a table, so that I am not bored playing alone
BT-02	Tutorial	3	As a player, I want to be able to have a tutorial, so that when it is my first time I know how to play

AV-01	Ad placement	1	As a admin, I want to be able to find places and times to put ads, so that I can make money and advert my product.
AV-02	Ad rotation	3	As a admin, I want different ads to play in rotation, so that I can get more ad sponsors
AM-03	chat logs	3	As a admin, I want to store user messages to look back on there account, so that I can moderate the site.
AM-04	remove	1	As an admin, I want to remove a registered user from the system so that I can manage accounts and remove users when necessary.
AM-05	Admin Console and Management	8	As an administrator, I want a centralized admin console where I can manage users, review reports, monitor games, and access administrative tools, so that I can maintain and manage the poker platform efficiently.
UP-01	Profile Creation and Management	5	As a registered user, I want to create, view, and edit my profile, including my username, avatar, chips, and player statistics, so that I can personalize my account and track my information.
UP-02	View Other Players' Profiles	3	As a player, I want to click another player's username from the leaderboard, friends list, or poker table to view their profile, so that I can see their statistics and compare my performance with theirs.
	Total:	57	

Table 2: Functional requirements as User Stories.

3.3 AI Analysis on Functional Requirements

Used Copilot. Prompt - what are some requirments needed for users when using a poker app. Second Prompt - it is not a poker game that uses real money and think of situations for users and admins or others.

Valid

- Being able to report other users important so admin can be called and view chat logs, we should also have chat logs as a user story for admin.
- We should be able to report other issue to the admin as well
- able to invite friend to a poker table or create a poker table.
- admin being able to remove inappropriate usernames.

Invalid

- Age requirement, it is not real money, we do not need it.
- Strong password and 2 factor authentication, not needed in our system. Users sign in with Google, so we never store passwords, and Google already provides strong passwords and 2FA

4 System Design

4.1 Architecture

We were thinking of using a layered architecture. The front end would show the pages and let players use the game (with the buttons, from our project stories). The game layer will handle player actions, turns, cards, and bets. The data layer will save and load the information. And we will keep the parts separate so that the code will be easier to work through and understand between us (of course, we will use easy to understand

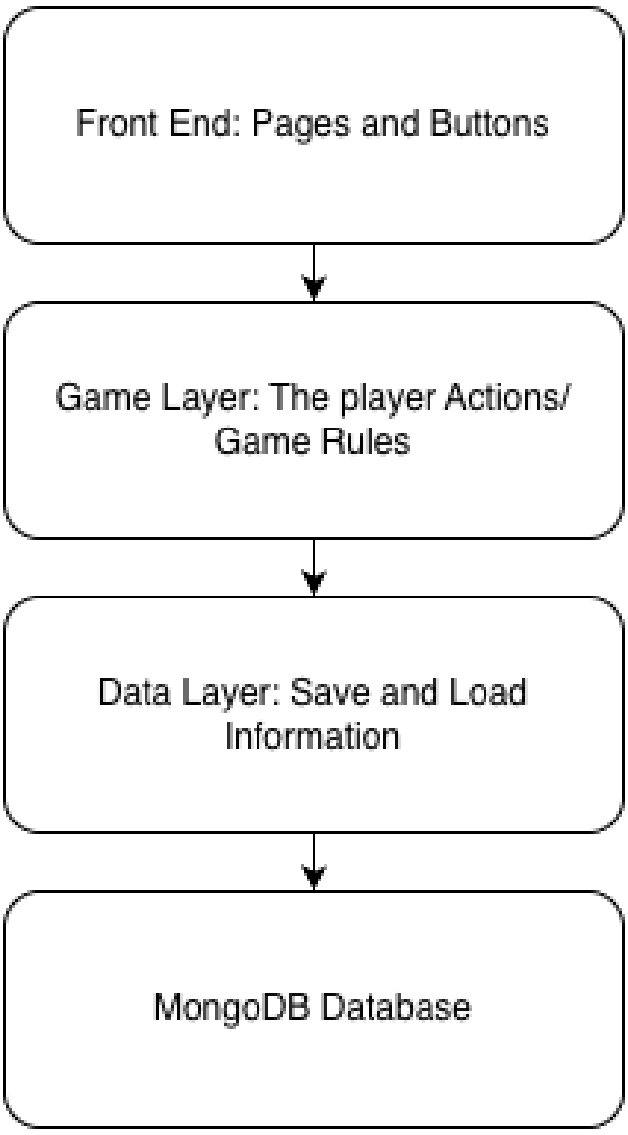


Figure 1: Enter Caption

function names to understand what the task of each task is). We will use on Java/ spring boot application and one MongoDB database. We will develop and test the program using local host on our laptops. Our first versions will be able to support small groups of players and a few tables. (reason we want to do this is too see if we are able to get the basis of game down, test multiple tables running at the same time, and see if the necessary functions are working). Our main modules will be users and accounts, lobby and tables, poker game, social features, player stats, and admin tools. These will handle profiles, joining games, gameplay, friends and chat, saved results, and account management. We want to focus on getting the basic features working before adding extra like bots, ad systems, etc.

4.2 Diagrams

4.3 Technology

We are going to choose Java as the main programming language for the back end because it is the language that all members of our team are most familiar with. This will reduce the learning curve and allow us to read each other's code better. We can then spend our time on the harder parts of the program like the game logic and the real-time multiplayer.

The best framework we found for building the server is the Spring Boot framework. This provides us a built-in web server so the application will be able to run on localhost during development and then later be deployed to the client's servers. It also has modules for the other features our project needs, which will keep our technology choices consistent across our system.

For data storage, we will use MongoDB, a NoSQL document database. MongoDB stores data as JSON documents, which will fit our data well since records like a user profile or a finished hand will contain nested info. The Java server will communicate with MongoDB through Spring Data MongoDB. Its repository classes act as our Data Access Objects (DAOs). So, only the data layer will talk to the database. This ensures that we follow the information hiding principle. We will store Avatar images that are uploaded by users in MongoDB using GridFS. This is a feature of MongoDB that stores files.

The front end will be built using HTML, CSS, and JavaScript. HTML will define the structure of each page, such as the lobbies, profiles, and poker tables. CSS will apply the purple Stack Overflow branding and also the casino-style layout. Finally, JavaScript will handle user interaction in the browser. Since poker is a real-time multiplayer game, regular web requests wont cut it. When one player bets, every other player needs to see it immediately. Our strategy to solve this will be to use WebSockets through Spring WebSocket. This will keep an open connection between each player's browser and the server. The server will publish game events like new cards, bets, folds, and chat messages to every player in that table.

A framework that will be new to all of us is how we will handle user authentication. Since we were asked to do this through Google authentication, we will handle it with Google OAuth 2.0 through Spring Security. This will let registered users sign in with their existing Google accounts. Because of this, our application wont need to store passwords and Google will handle account recovery. This also covers the client's request for password recovery and two-factor authentication, since Google accounts already provide both.

For testing, we will use JUnit 5 to write the automated tests. This will be done to help to test out the game engines like dealing cards, betting rounds, deciding the winning hand, and more. Mockito will also be used to mock the database layer so the unit tests run quickly and wont depend on MongoDB. We will measure the coverage with JaCoCo, which reports both line and branch coverage. Finally, for full system tests, we will use Selenium to automate a browser and simulate players moving through the application

Finally, our project will use Maven to manage dependencies and build the project. We will use IntelliJ IDEA as our development environment, and GitHub for version control. GitHub Projects will remain used as our Scrum board to track user stories, tasks, and other assignment progress.

4.4 Coding Standards

Overall, we will try to maintain the same coding standards we have had with our PAs at Loyola. Each larger coding file must have a header that explains it's purpose, author, and any input/output. Each function aside from a main() should also have a header that explains its purpose, input, and return. Each person should try to have meaningful variable names so someone else can understand it. Code should also have appropriate whitespace and avoid overly long lines.

When it comes to using Git, our team will have accurate commit messages and commit regularly after milestones. This will match the XP programming "baby steps" principle. We will also not commit broken code. It at least must compile. We will make sure not to commit any MongoDB connection strings or any other secret information.

Our Java code will follow the standard Java conventions. Variables and methods use camelCase and classes use PascalCase. In MongoDB, collection names will be singular and lowercase, and field names will use camelCase. Fron-end JavaScript and CSS will be split into seperate files for each page instead of placing all the code in one page.

For testing, every new class with game or business logic must include JUnit tests, and the database will be mocked with Mockito so the tests run quickly. Our overall test coverage, measured with JaCoCo, must stay above 60%. Since the poker game engine decides who wins chips, we will aim for a higher coverage of

80% for that module. All tests must pass before code is merged and when a bug is found, we will first write a test that reproduces that bug.

4.5 Data

We will use MongoDB to store our data in collection. Each of the records will have an ID. We will then use those IDS to connect the records, such as linking messages to players who sent it. The user collection will include the user ID, the good account ID, the username, the avatar file id, the balance of the chips, the wins, the losses, the number of hands played, and the fold count. It will also store basic settings and whether the user has finished the tutorial (earlier in the user stories). A role field will show whether the user is a player or admin, a ban end date will show whether the account is banned. We will calculate win percentages from the saved results. The tables collection will store the table id, table name, owner id, bet limits, player limit, and the ids of the players at the table. It will also show whether the table is public or private. During a game, the server will keep track of the current cards, bets, and turns. The hands collection will store a simple record of each finished hand. This will include the hand id, table id, player id, the winners, chips won or lost, and the time the hand ended. The friendship collection will sort the sender id, the receiver id, and request status. The status will show whether a friend request is waiting or accepted. The message collection will store the send id, msg text, and time sent. A table msg will have a table id. A private msg will have the receiver user id. the reports collection will store who sent the report, which player was reported, the reason, and the time . A status field will show whether an admin has handled it. Guests will have to use a temporary session. Their chips and stats will not be saved after they leave. Our first version will use virtual chips only.

4.6 UI Mocks

The following mockups show the main screens of our application. They follow the client's purple branding (#bf40bf) and use the same navigation bar on every page so the interface stays consistent. Placeholder boxes mark where the logo, avatars, and advertisements will go, and values in braces (such as {Username}) will be filled in with the player's real data.

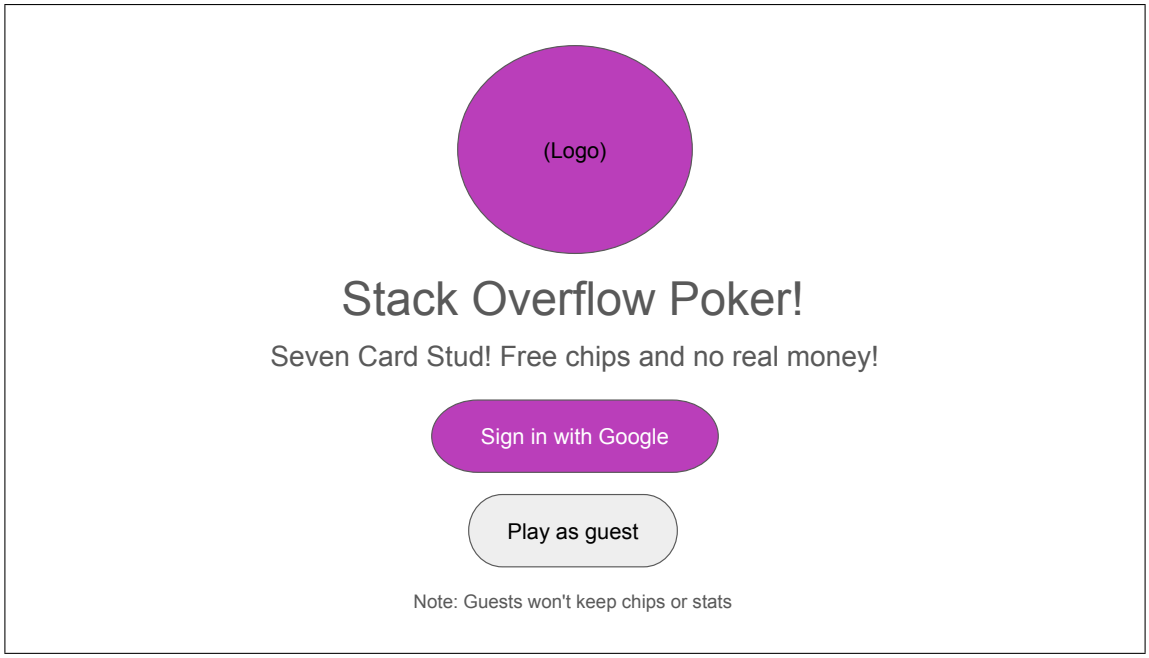


Figure 2: Welcome screen: players sign in with Google or play as a guest.

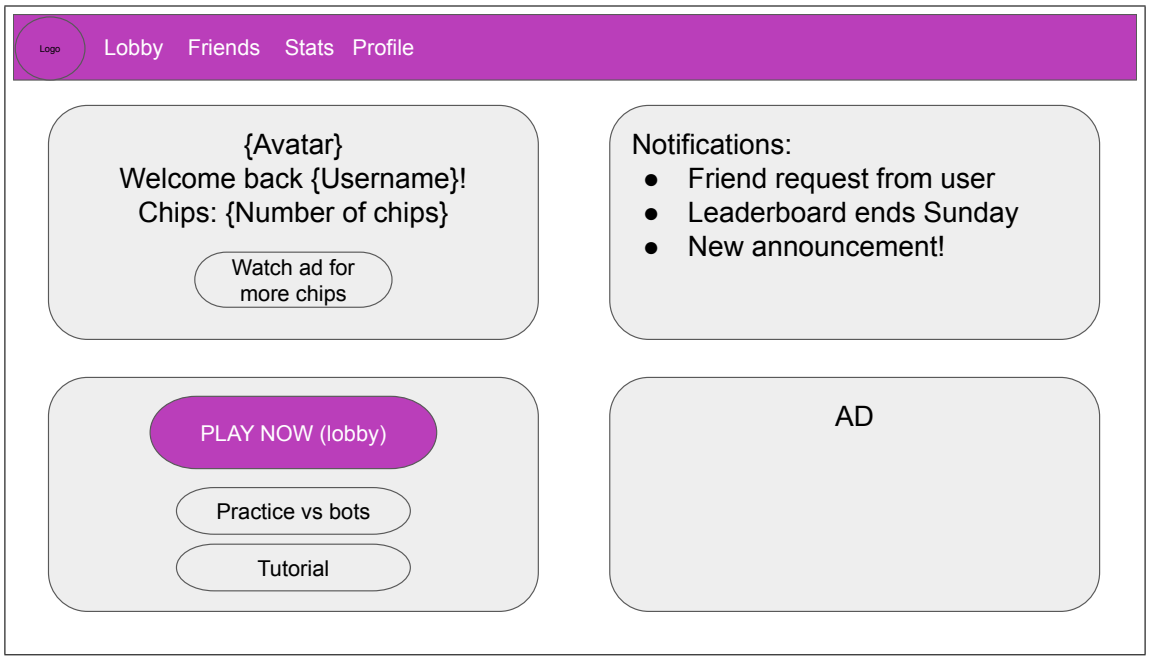


Figure 3: Home screen: chip balance, notifications, and quick access to games, practice bots, and the tutorial.

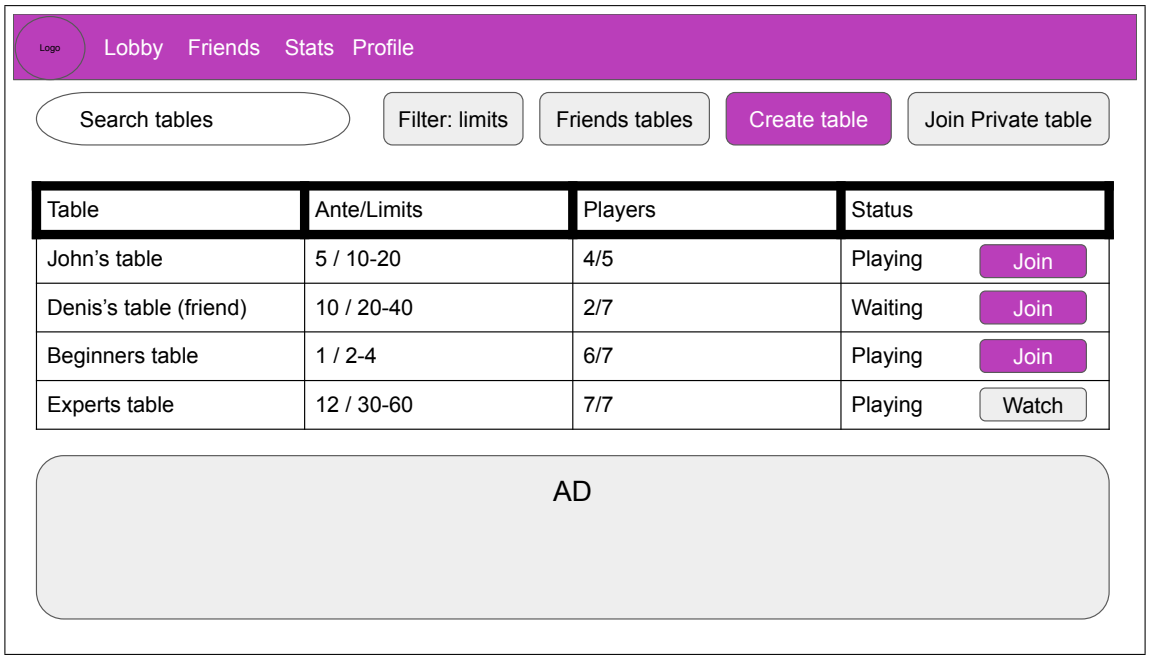


Figure 4: Lobby: players search, filter, create, and join public or private tables.

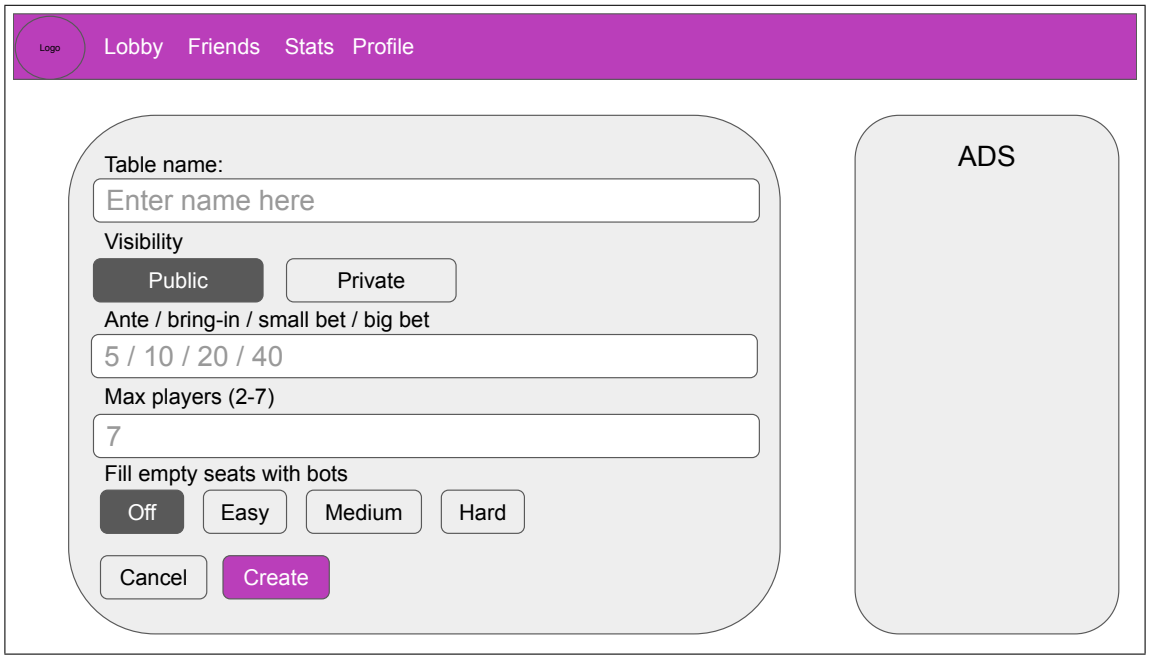


Figure 5: Create table form: table name, visibility, betting limits, maximum players, and bot difficulty.

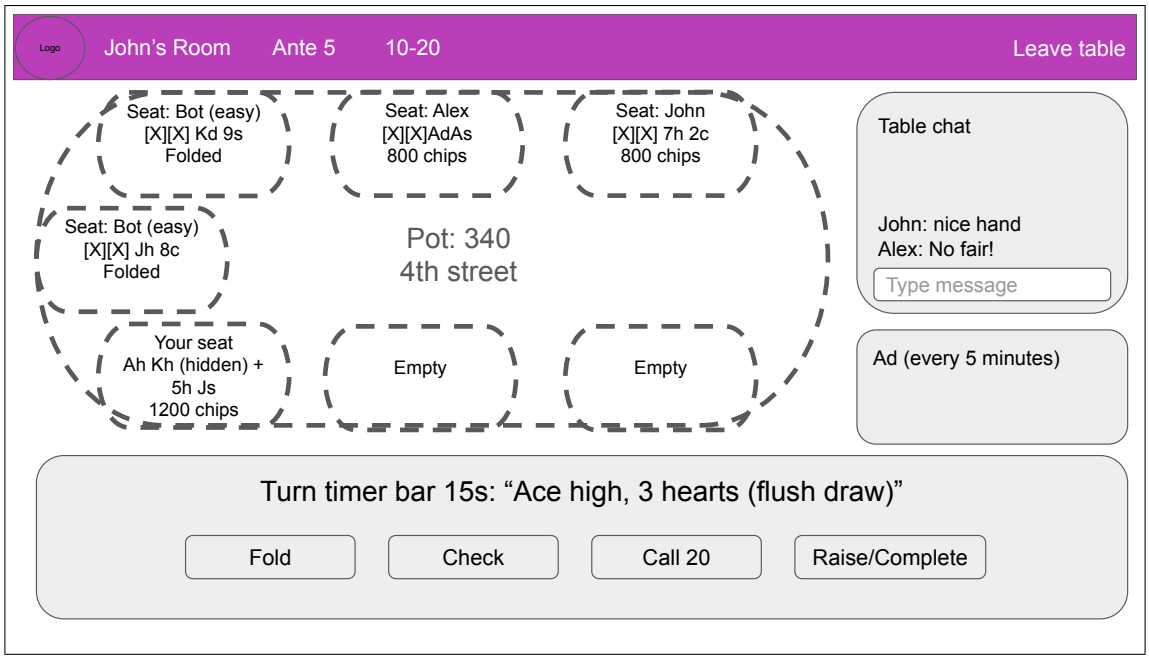


Figure 6: Game table: face-down and face-up cards, the pot, the turn timer, the optional hand helper, betting actions, table chat, and an advertisement.



Figure 7: Another player's profile: players can add a friend, send a message, or report the player.

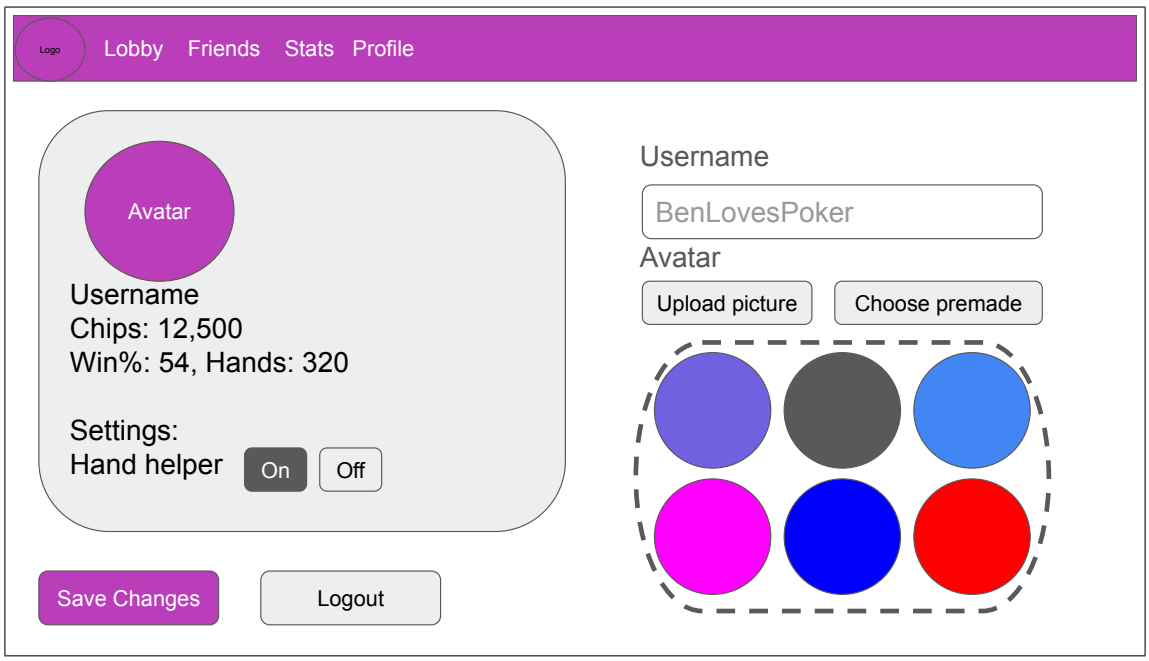


Figure 8: Own profile and edit form: change username and avatar, toggle the hand helper, save changes, or log out.

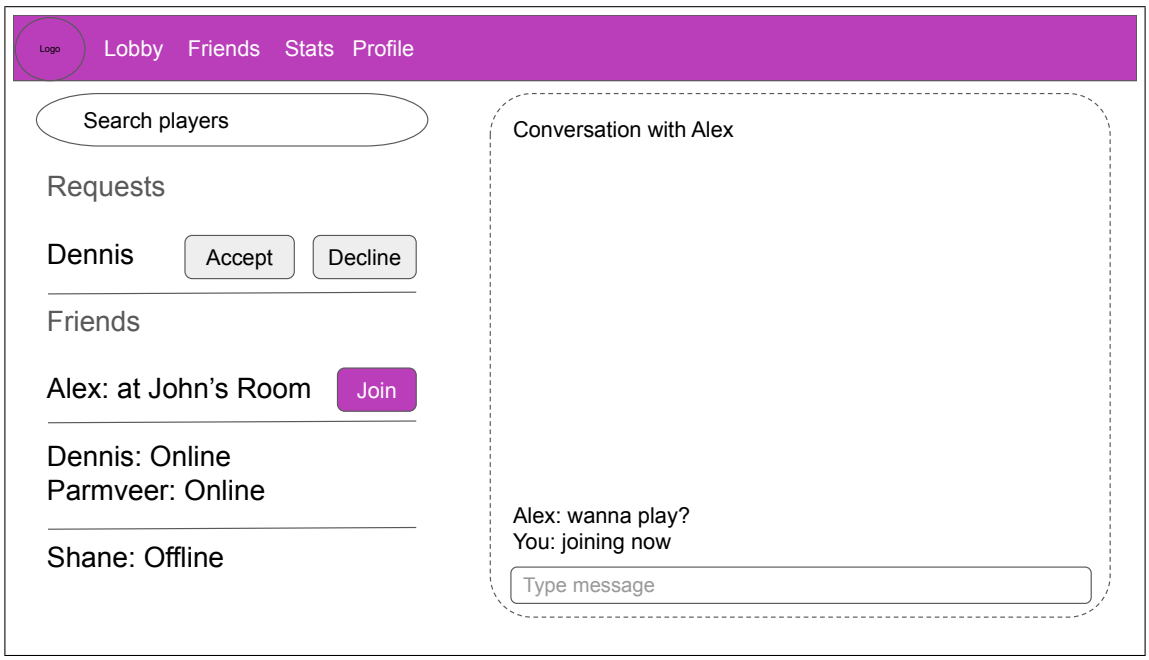


Figure 9: Friends: accept or decline requests, see friends' status, join a friend's table, and send direct messages.

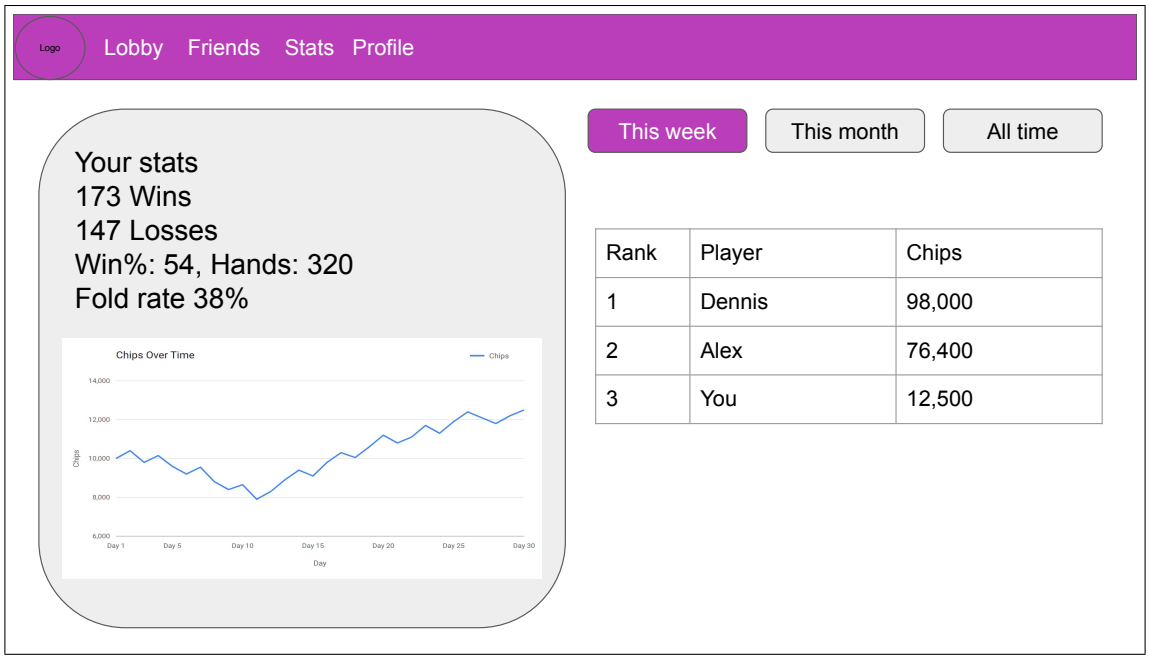


Figure 10: Statistics and leaderboard: personal stats and weekly, monthly, and all-time rankings.

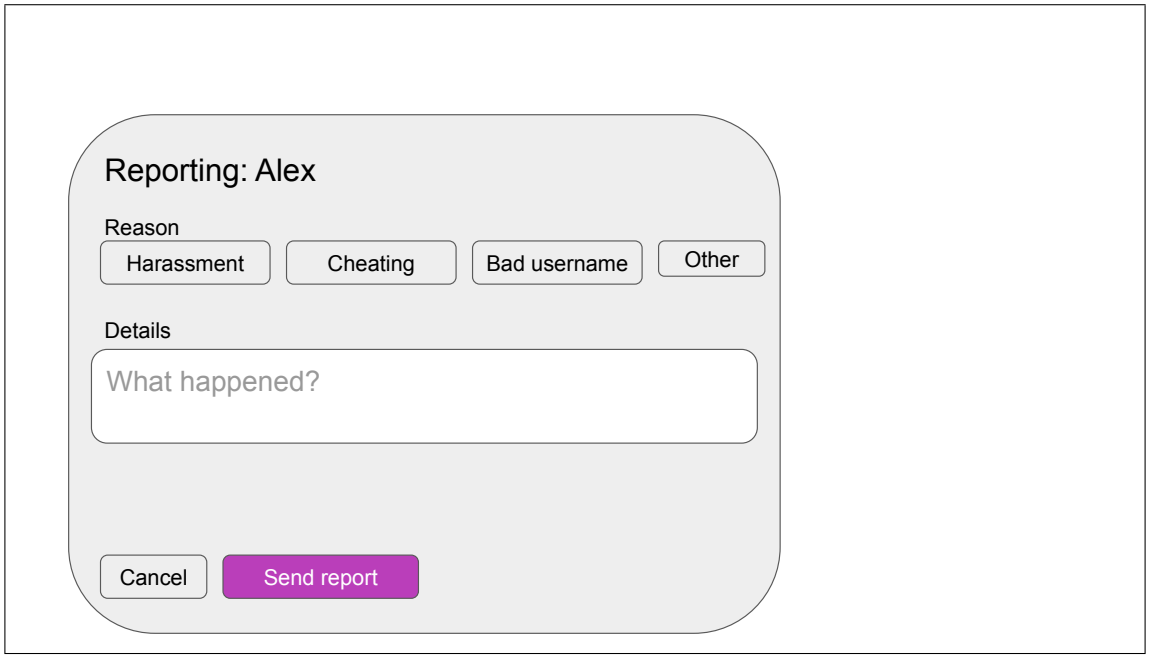


Figure 11: Report player form: players choose a reason and describe what happened.

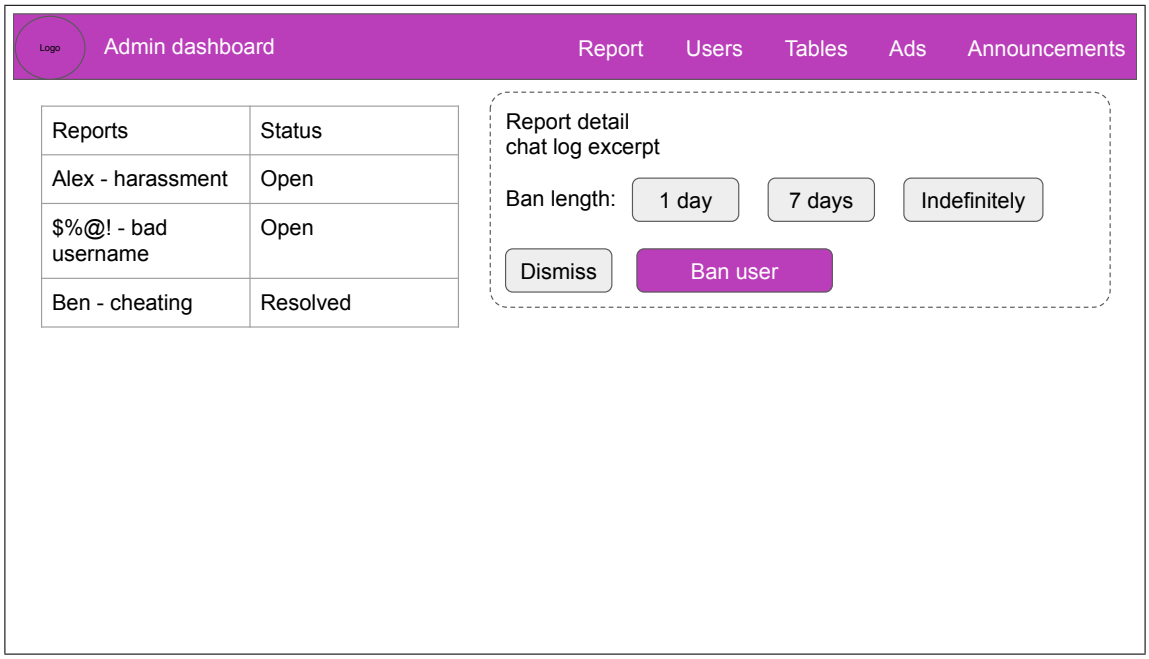


Figure 12: Admin dashboard: administrators review reports and chat logs, then dismiss the report or ban the user.

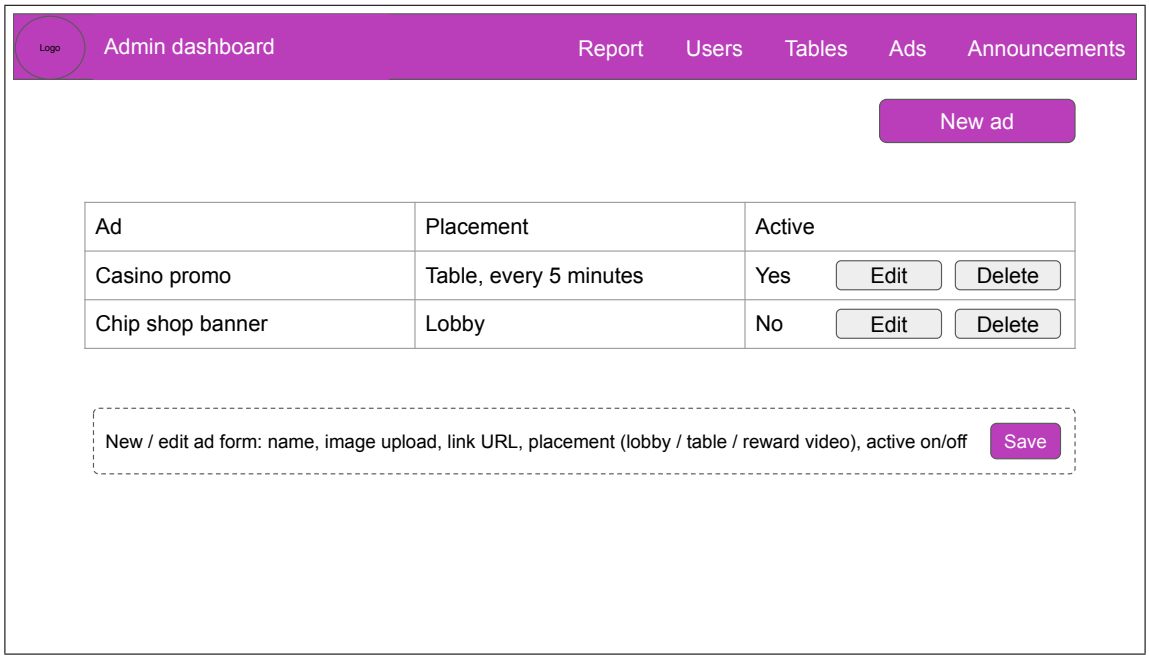


Figure 13: Admin advertisement management: create, edit, and delete ads and choose where they appear.

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 3 shows the iteration planning.

			Points	
It.	**Dates**	**Stories**	**Planned**	**Done**
1	01/01 - 02/01	S1 Story Example, S2 Story Example 2	07	05
2	02/01 - 03/01	S3 Story Title, S4 Story Title, S5 Story Title, S6 Story Title	17	15
3	03/01 - 04/01	S7 Story Title, S8 Story Title, S9 Story Title, S10 Story Title, S11 Story Title	21	20
4	04/01 - 05/01	S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title	19	19
5	05/01 - 06/01	S16 Story Title, S17 Story Title	06	06
		Total:	70	65

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

Ben Ackerman: Friends list, get friend request, send friend request SF-01, SF02, SF03

Shane Ford: Profile stuff (ST-01 AND 02; AM-01)

Dennis: Lobbies, tables, creation (LT-01:LT-02:LT-03)

Parmveer: deal cards, gambling (PG-01, PG-02)

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

Work(Denis Moroz).

For Lt-01, worked on the lobby so the players can see which public tables are available. I added backend code that gets table info from MongoDB and displays the tables names, status, and number of players. Private tables are to be nnot shown in the public lobby. For Lt-02, added the option to join a table using the username. Before adding a player, the backend would check that the table exist, is waiting for the playes, has an open seat, and doesn't have that username already. I also connected the public and private table joining to the table page. Private tables could be joined using a shared table ID as an invite code. For Lt-03, I connected the form to the backened so a player can create a public or private table with a name, player limit, ante settings, and bot difficuly setting. The new tables was saved in MongoBD, then added a table page based on our UI mockup. It loads the selected table, shows the players, and empty seats, and refreshes the player list.

Work(Ben Ackerman). For SF-01, SF-02, and SF-03, I worked on the friend system. Before the friend features could work, users needed to be stored in the database. Because of this, I first set up the connection

between our Node.js and Express server and our MongoDB Atlas cluster with the .env file. Then, I added the users collection where each user has a username, a friends list, and a list of pending friend requests. We haven't added Google sign in yet, so I made a temporary login where the player types a username. If the username already exists, it will log into that account. If not, a new account is created. For SF-01, a player can type another player's username on the Friends page that I made. Te they press Add Friend to send a friend request. The backend will then check if the the other player exists, is not the current user, is not already a friend, and has not already been sent a request before sending it. For SF-03, the receiving player will see the request under Requests and can accept or decline it. Accepting this requests will add both players to eachother's friends list and declining will remove the request. Additionally, if two players were to send eachother a request, the second one counts as accepting the first. And then for SF-02, the Friends page loads the logged in player's friends and pending requests from MongoDB and displays them. I also wrote unit tests for the database code and the controller for all threes tories. I made the base website design and branded the technical task, which made the shared purple navigation bar, stylesheet, and a page for each screen from our UI mockups. This task is still in progress because the team and I decided the design should be further improved before it is finished.

Time spent:

- SF-01 through SF-03 were built together and took about 9-10 hours in total.
- Setting up MongoDB took around 4-5 hours, including configuring the cluster's network access so the rest of the team could connect.
- Creating the HTML base took about 2-3 hours.

Work(Shane Ford). During this iteration, I successfully completed two user stories for our StackOverflowPoker project: ST-01 (Player Statistics) and ST-02 (Leaderboard). For ST-01, I implemented a feature that allows users to view their total wins, losses, and winning percentage. This involved updating the MongoDB user schema, creating a controller to calculate player statistics, adding an API endpoint, developing the statistics webpage, and writing automated tests using Jest. For ST-02, I created a leaderboard that allows users to compare their performance with the top players based on their total wins. I developed the database query to retrieve and rank players, created the leaderboard controller and API endpoint, designed the leaderboard webpage, and added navigation so users could easily access it. I was responsible for implementing and testing both stories, and I successfully completed all the required tasks for each. Both features were tested and completed during this iteration.

Work(Parmveer Basara) During this iteration I was in charge of two user stories for gambling and dealing cads. I was able to get the mechanics of them working but wasn't able to completely implement them on the website so I wasn't able to really complete them. I had created functions to be able to create a deck of cards and shuffle them. Also functions to be able to deal two cards at the beginning and one at the end. Created functions to be able to change the number of chips a player had. I had also worked on some of the table but very little mostly just the form to create it and fucntions in the DOA and controller to leave the table.

5.2.3 Testing Coverage

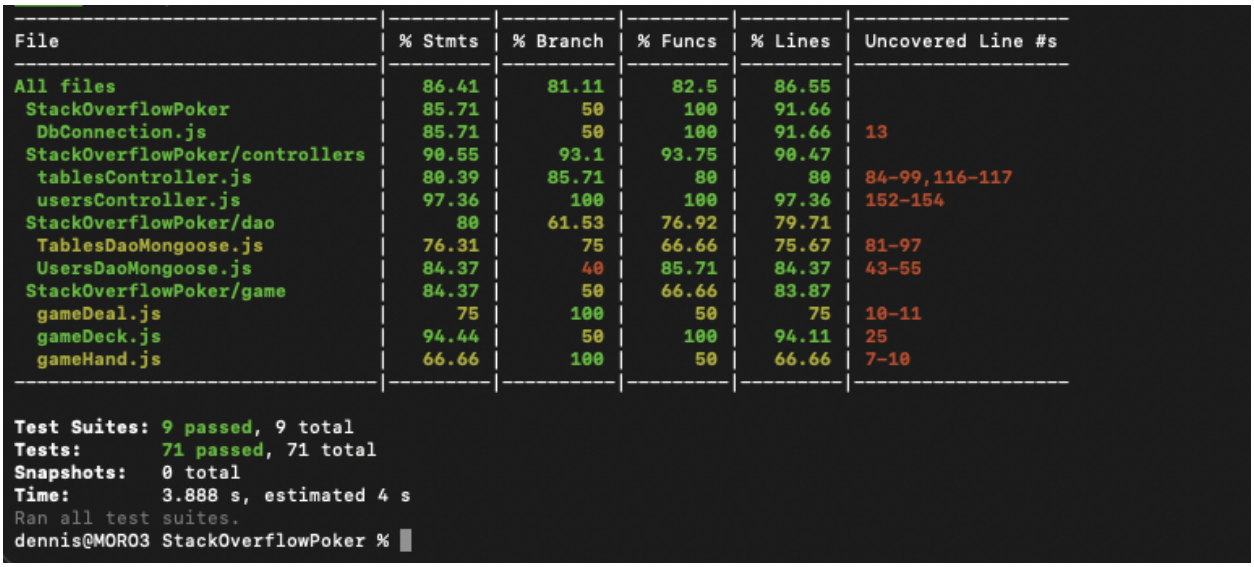
[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

For this iteration, we used Jest to test our project and make sure the main funcitons where working correctly. We tested different features, include creating and joining tables, user accounts, friends, player stats, leaderboards, and some game functions. We also tested different situations, such as invalid inputs, duplicates, full tables, and database erros. We wanted to check how the program would handle both successful and unsuccessful actions.

After running **npm test – –coverage**, all 71 tests passed across the 9 test suits with not failures (good results in our opinion).

The overall statement coverage was 86.41%, branch 81.11%, function 82.50%, and line was 86.55%. (also good results in our opnion, but could use improvement).

We also manually checked some of the website functions, such as creating a public table and joinging through the lobby. However, some front end features and connections still need more testing. I attached a screenshot of our jest test results as well in git. File path: CS-482-Project/docs/coverage.png



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	86.41	81.11	82.5	86.55	
StackOverflowPoker	85.71	50	100	91.66	
DbConnection.js	85.71	50	100	91.66	13
StackOverflowPoker/controllers	90.55	93.1	93.75	90.47	
tablesController.js	80.39	85.71	80	80	84-99,116-117
usersController.js	97.36	100	100	97.36	152-154
StackOverflowPoker/dao	80	61.53	76.92	79.71	
TablesDaoMongoose.js	76.31	75	66.66	75.67	81-97
UsersDaoMongoose.js	84.37	40	85.71	84.37	43-55
StackOverflowPoker/game	84.37	50	66.66	83.87	
gameDeal.js	75	100	50	75	10-11
gameDeck.js	94.44	50	100	94.11	25
gameHand.js	66.66	100	50	66.66	7-10

```
Test Suites: 9 passed, 9 total
Tests:       71 passed, 71 total
Snapshots:   0 total
Time:        3.888 s, estimated 4 s
Ran all test suites.
dennis@MORO3 StackOverflowPoker %
```

Figure 14: Jest Testing Coverage

5.2.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

Denis Moroz: During this iteration, I learned more about the front end, backend, and database connecting with each other. One of the mian challanges was making sure creating and joining the tables worked correctly accross different files (especially when we all have access to the git repo and we are all editing it, the git pulls/mergers where annoying to deal with). I had to at one point resort to editing the files over the online github so I wouldn't get the merger conflicts or work late at night. One thing to improve would be better communication when working on same files, and introduce a system of file sharing.

Ben Ackerman: This iteration for me was a great recap session of my class in Web Programming. I reused many strategies from that class and this was a huge help with the creation process of the application like the MongoDB and the JavaScript and HTML. I knew that setting up the MongoDB on other user's computers would be a great challenge. Me Shane worked hard to get it to work on his potterbox but we ended up having to resort to everyone launching it on their windows terminal in their computer. I am glad we all got in properly and this was a great learning experience. In the future, we will definitely try to get more people into MongoDB earlier so people have more time to work on their portion of the project. I noticed when testing out my portion of the project that I did have a high percentage for my tests but I actually did not have for all other files so I had to adjust. Overrall, it was great learning how to add things to MongoDB that will send another user data through their friends lists with requests. I definitely realize after seeing todays presentations that we should use AI more for the UI to make it look nicer and it will be much easier to implement everything once users are fully functional.

Shane Ford: During this iteration, I faced several challenges while implementing both user stories. One of my biggest difficulties was making sure the different parts of the application worked together correctly, especially when connecting MongoDB, JavaScript controllers, API routes, and the frontend. While working on ST-01, I encountered a syntax error in my MongoDB schema caused by a missing comma, which prevented my Jest tests from running. I also had to fix an issue where I used an incorrect variable name in my recordGameResult function. Another challenge was understanding Git branches and why some of my commits were not immediately appearing on GitHub after I made changes. During ST-02, I also encountered an issue with the req parameter in my controller. Although these problems slowed down my progress, I was able to troubleshoot and resolve them. I learned more about how MongoDB stores and retrieves information, how JavaScript functions and Express controllers interact, and how to manage Git branches and commits more effectively. One of my biggest takeaways was the importance of testing code frequently rather than waiting until the end of a task. Moving forward, I plan to check my code for errors more often, make smaller commits, and verify that I am working on the correct Git branch before pushing changes. Overall, this iteration helped me improve my debugging skills and gave me more confidence working with full-stack JavaScript applications.

Parmveer Basara In this iteration I wasn't able to fully implement my work onto the website for functional use. A overall unsatisfiable amount of work to bring the website to reality. I was able to complete the mechanics and but with nothing to show for the user it does not mean anything. I have to put more work into the website and be able to show it to the users. In the next iteration I will have completed all my tasks properly. In this iteration I did learn things about databases when I created the functions to leave the table. When the database would store the information like username with a table and how to remove that user from the database from a specific table.

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

Ben Ackerman: Remove friend, Well Polished UI, Technical task for implementing users fully with google authentication SF-04: 1 point Well Polished UI: 2 points User Management: 2 points

Shane Ford: Making user stories for profile creation and admin console as needed AM:1-4,

Parmveer Basara: Finish PG-01, PG-02 and Work on SF-5 SF-6

Denis Moroz: BT-01 and BT-02

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.] **Ben Ackerman:**

Ben Ackerman For this iteration I worked on three things: removing friends (SF-04), finishing the website design (TT-01), and replacing our temporary username login with the real Google sign-in (TT-03).

For SF-04, a player can now open a friend's profile and press 'Remove Friend'. After confirming, the backend removes each player from the other's friends list and the Friends page will show a message saying who was removed, The backend will also check that both players exist and are friends before changing anything.

For TT-01, I redesigned every page to match our UI mockups more closely. It has a shared purple navigation par, rounded cards, a more casino like lobby, and a poker table view. I also worked on the Stack Overflow logo for it to be a falling stack of poker chips. Then after I finished my main things I added a dark mode that players can turn on from the Profile Page. It changes the whole site to be darker and is remmebered on the device

TT-03 was the hardest one. In Iteration 1, anyone could log in with a username, and the username was passed from page to page in the URL, so a player could ac as someone else if they were to just change the link. I have replaced this with a Google sign-in using Passport and the Google OAuth 2.0. The server will now keep a login session in MongoDB, so it always knows who is making each request. The first time a player signs in, they choose their poker name. There Name must be 3 to 20 characters, it is checked live as the player types, and it can not be reserved words like admin and also can't match an existing name. If someone plays as Guest, this still works and they are given a temporary name like Guest-4322 that lasts for their session. Guests can join tables but cannot add frends or maintain stats

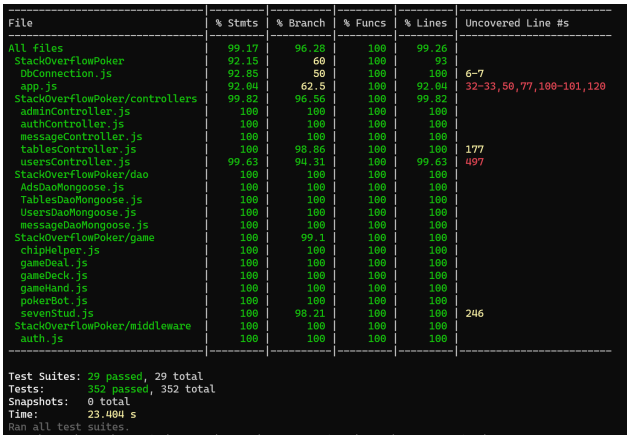
With the sessions working, I secured the friend routes and the table join and leave routes so they will act as a signed in player instead of just trusting a name sent from the browser. I also removed the old username login and updated all the old pages to the new system. A script had to be written that clears all old users, tables, and sessions. Time spent:

- TT-01 during this iteration: 3-4 hours
- SF-04 1 hour
- TT-04: 4-5 hours

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

For Iteration 2, we continued to use Jest for our autmoated testing. After running npm run coverage, all 352 tests passed across 29 test suites with no failures which is way up from 71 in iteration 1.



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.17	96.28	100	99.26	
StackOverflowPoker	92.15	60	100	93	
DbConnection.js	92.85	50	100	100	6-7
app.js	92.04	62.5	100	92.04	32-33,50,77,100-101,120
StackOverflowPoker/controllers	99.82	96.56	100	99.82	
adminController.js	100	100	100	100	
authController.js	100	100	100	100	
messageController.js	100	100	100	100	
tablesController.js	100	98.86	100	100	177
usersController.js	99.63	94.31	100	99.63	497
StackOverflowPoker/dao	100	100	100	100	
AdsDaoMongoose.js	100	100	100	100	
TablesDaoMongoose.js	100	100	100	100	
UsersDaoMongoose.js	100	100	100	100	
messageDaoMongoose.js	100	100	100	100	
StackOverflowPoker/game	100	99.1	100	100	
chipHelper.js	100	100	100	100	
gameDeal.js	100	100	100	100	
gameDeck.js	100	100	100	100	
gameHand.js	100	100	100	100	
pokerBot.js	100	100	100	100	
sevenStud.js	100	98.21	100	100	246
StackOverflowPoker/middleware	100	100	100	100	
auth.js	100	100	100	100	

Test Suites: 29 passed, 29 total
Tests: 352 passed, 352 total
Snapshots: 0 total
Time: 23.404 s
Ran all test suites.

Figure 15: Enter Caption

The overall coverage was 99.17% of statements, 96.28% of branches, 100% of functions, and 99.26% of lines. We consider this very good: it is well above our coding standard's 60% minimum, and the poker game engine, which decides who wins chips, is at 100% line coverage, above our 80% goal for that module.

We made two changes to how we measure testing for this iteration. For the first one, it was moving every test file out of the seperate 'tests' folder so each one sits next to the file it tests. Second, we made Jest count every server file in the coverage report and not only files our tests would happen to load. Before this change, files without any tests would be left out of the total which would make our look higher than it really is.

The new tests cover the full engine, game routes, chip balances for both guests and users, message ads, admin tools, google sign in, and guest sessions.

Not everything is covered with the automated tests though. The front end JS in our web pages is chacked manually in the browser instead and server.js and our one time setup stuff are excluded because they only start the server or change the database.

5.3.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

Ben Ackerman: The hardest challenge I overtook this iteration is the Google sign-in, since I have never used OAuth before. I had to setup a Google Cloud project, then understand how Google sends the player back to our serber, and make sure that the client secret and session secret stayed in our .env file so its never on GitHub. Switching from the username login to sessions also had to toucn on pretty much every other part of the siste since every page and multiple routes depended on the username to be in the URL. I split this into database changes, the sign in flow, choosing a name, and then the front end.

Changing how login works affected my teammates' dode, when I pulled their changes I had to make sure that everything still worked. I am glad I made my changes early in the iteration so my teammates had time to give me their emails for the Google login and to accomodate their parts.

I learned how a coverage number is misleading with our report looking fine until I had Jests look in every file where it showed files with no tests.

I used AI tools this iteratiion to help with our UI design and tests. This helped reduce the workload tremendously and to add an extra layer of touch I would not have added otherwise. I am happy that our UI is now on par with those groups around us.

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]